DH2323 LAB 1

Set-up and Introduction to 2D and 3D Graphics

with Processing

There are three main tasks in this lab:

1) Download and set up *Processing*.
2) Interpolation (see Figure 1, left).
3) Starfield (see Figure 1, right).

Each build on the previous one.

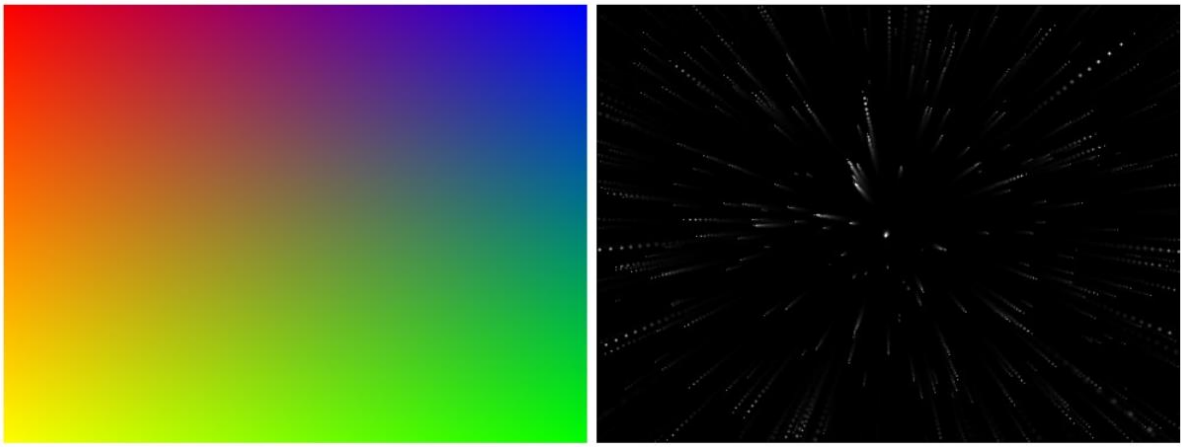

Figure 1. The output from this lab for (left) Task 2: Interpolation and (right) Task 3: Starfield.

Note: We provide a skeleton program which you can (but do not have to[1]) use as a basis for the labs. It uses the Java variant of Processing (https://processing.org/) to create a surface on the screen that you can draw on and then fills it with a constant blue color. If you do not want to use it and have another way to plot pixels in a window on your screen, you can proceed straight to Task 2: Interpolation.

[1] You can use any media library and programming language that you like. If you do that, we suggest that you use something you are already very familiar with.

Task 1: Running the Lab1 skeleton code

Our first task will just be to get the skeleton code building into an executable ***on your platform***.

When you run the skeleton code successfully, you should see the following screen:

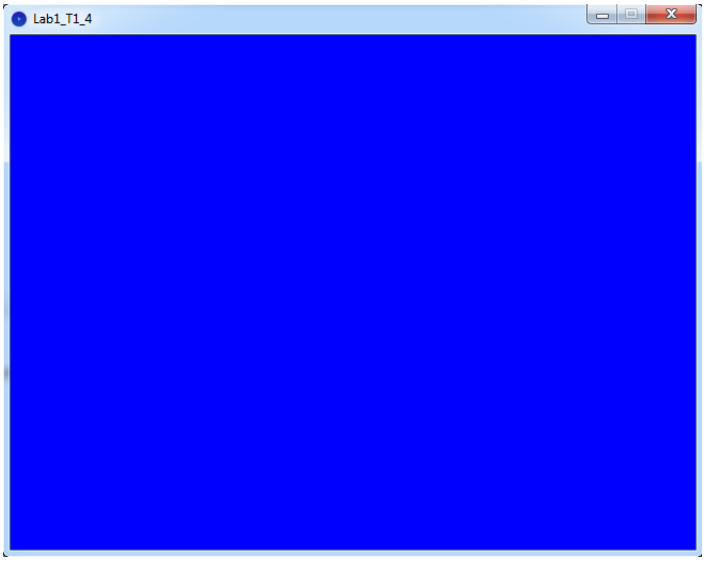

Figure 2: The expected output from the Lab1 skeleton. Impressive, eh?
No… well, just keep at it.

1.1 Download the Lab1 skeleton code

The main source code file is *lab1.pde*. This file contains programming commands the Java variant of Processing (Javascript and Python variants are available too) to create a window on the screen and draw some pixels to it. You can find out more about Processing here: https://processing.org/environment/

Task 1.1:

Download the latest version of Processing for your OS from https://processing.org/download and extract it to a directory of your choice.

Next, locate the folder on your system where Processing projects (called 'sketches') are stored. To find the default location, which varies depending on the OS you are using, select the "Preferences" option from the File menu (or from the "Processing" menu on the Mac) and look for the "Sketchbook location."

Task 1.2:

Download the DH2323 skeleton code for Lab1 from the DH2323 Canvas page. Then go to the location from Task 1.1. For me, this was the "Documents/Processing/" folder. Create a directory for your task (named for example "Lab1") and copy the *lab1.pde* file into it.

Task 1.3:

Double click on *lab1.pde* file or open it in Processing using the "File->Open" option. You should see a screen similar to that shown in Figure 3.

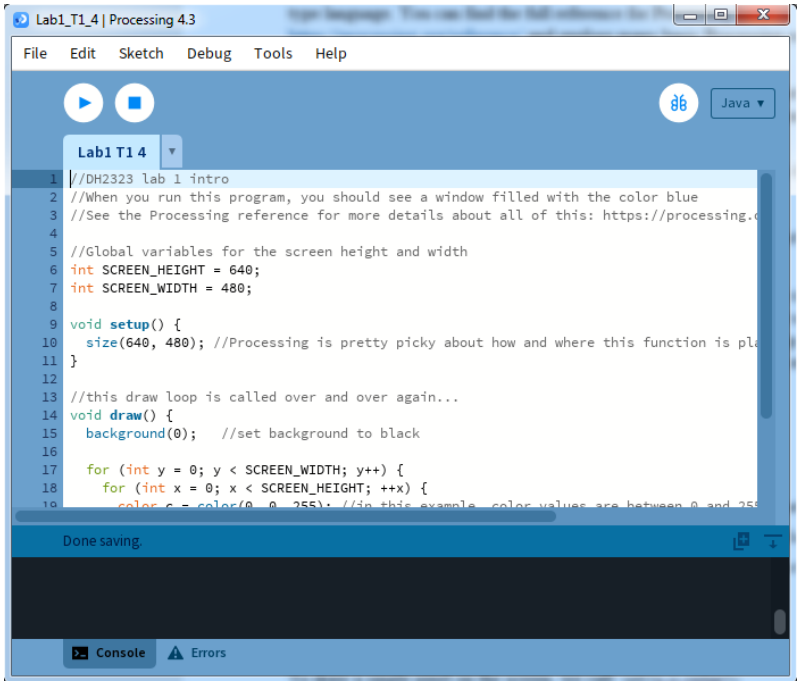

Figure 3: The *Processing* sketchbook environment.

Task 1.4:

Run the code by clicking the circular run icon and you should hopefully see the results in Figure 2 above. Bask in the warm feeling that you've saved a lot of time by doing the Processing version of this lab and did not have to wrestle with the build process…

Task 1.5:

Look at the code and experiment with the colours. You will see that it is written in a Java-type language. You can find the full reference for Processing here: https://processing.org/reference/ and explore many basic Processing examples here: https://processing.org/examples/

Processing includes some useful utility classes, such a *PVector*, but its primary use in this course is very simple: opening a window, processing keyboard input and drawing a pixel to the screen.

The initial code uses Processing to create window/screen with 640 x 480 pixels:

```
size(640, 480);
```

When the program is executed, it starts by running the `setup()` function. First, the screen is cleared to black before we draw to it, we call: `background(0);`

Then, the nested *for* loop moves to each position on the screen colouring each pixel in. Specifically, it moves horizontally across the first row of pixels on the screen, from left to right, colouring each, until it reaches the end of the row in the inner loop. The outer loop moves

vertically down[2] the screen, row by row, until all rows of pixels on the screen have been completed.

```
for (int y = 0; y < SCREEN_WIDTH; y++) {
  for (int x = 0; x < SCREEN_HEIGHT; ++x) {
    color blue = color(0, 0, 255);
     //in this example, color values are between 0 and 255 (rather than 0.0 and 1.0)
    set(x,y,blue); //set the color of the pixel at (x,y) to blue
     //note that set() is slow - there is also a pixels[] function is you want to be
     //fancy about it
  }
}
```

To draw a single pixel on the screen, we call: `set(x,y,color);`

This function takes the position, in x and y coordinates, as well as the color of the pixel to be drawn on the screen, as parameters. You can find more information about colors in processing here: https://processing.org/tutorials/color

As the main task here, change the colour of the screen to a colour of your choice. Experiment with drawing different colours until you understand the colour model. You can try to fill the screen with some of the following colours: red, yellow, green, cyan, blue, magenta, black, white and different intensities like dark red or pink.

Task 2: Interpolation

Once you have a basic idea of how to manipulate colours, it's time to move on to *interpolation*. Interpolation is all about creating estimates to construct new data points based on a range of already known data points. It has a broad range of uses in computer graphics – here we will investigate how we can use it to create smooth changes between different colours automatically for us.

Task 2.1:

Linear Interpolation. Your task in this part of the lab is to fill the surface with the rainbow effect seen to the left in Figure 1. You should specify a color for each corner of the surface and then interpolate these in between. Linear interpolation is often used in computer graphics programming. In the rasterization lab you will use it a lot. That is why we want you to implement some simple linear interpolation already in this first lab. You should write a function that fills the FloatList r with values linearly interpolated between *a* and *b*:

```
void Interpolate(float a, float b, int numResults, FloatList r){
```

[2] It moves down the screen as y increases here, because *Processing* has the origin (0,0) at the **top-left** with the positive y axis pointing downwards.

```
        // Your code goes here
}
```

Then in the *setup()* function:

```
FloatList result = new FloatList();  //create the a FloatList object to store the results
Interpolate(5.0, 14.0, 10, result);
        //interpolate between 5 and 14 with 8 intermediate values (10 in total when
        //we include the first and last values)
for (int i = 0; i < result.size(); ++i){
        print(String.format("%.0f ", result.get(i))); //print the results to the console
}
```

This should produce the output:

```
5 6 7 8 9 10 11 12 13 14
```

More information about FloatList is available here:

https://processing.org/reference/FloatList.html

You might need to treat the special case when the size of the vector is 1 with an if-statement, to avoid division with zero. Think about a suitable value to return in this case. You can write the test code above in the beginning of the main function.

If you need help figuring out how to do it, you can download the .pde source file for this Task from the Canvas page.

Task 2.2:

After you have the Interpolation from Task 2.1 working, make a new version that interpolates between two 3D vectors. In order to do this, you can use the Processing PVector class (https://processing.org/reference/PVector.html) and Java utility class ArrayList (https://www.w3schools.com/java/java_arraylist.asp).

To use the ArrayList class, you'll need to import it:

```
import java.util.ArrayList; // import the ArrayList class
```

and then create a new array of interpolation results:

```
ArrayList<PVector> resultVectors = new ArrayList<PVector>();
```

Your Interpolation function would then look something like this:

```
void Interpolate(PVector a, PVector b, int numResults, ArrayList<PVector> r){
        // Your code goes here
}
```

As in Task 2.1, you can start by adding the code above.

To test that your implementation produces the right output you can try to use it like this:

```
vector<vec3> result( 4 );
```

```
PVector a = new PVector(1,4,9.2);
PVector b = new PVector(4,1,9.8);
Interpolate(a, b, 10, resultVectors);
for (int i = 0; i < resultVectors.size(); ++i){
      PVector r = resultVectors.get(i);
      print(String.format("( %.0f, ", r.x));  //print the results to the console
      print(String.format("%.0f, ", r.y));  //print the results to the console
      print(String.format("%.0f )", r.z));  //print the results to the console
}
```

This should produce the output:

(1, 4, 9.2) (2, 3, 9.4) (3, 2, 9.6) (4, 1, 9.8)

Now you have a function that can be used to interpolate the most important quantities in computer graphics: 3D positions and colors, which we will both represent with 3D PVectors.

Task 2.3:

Bilinear Interpolation of Colors. Now that your interpolation works you should use it in the Draw-function to interpolate colors across the screen. First define the colors for each corner of the screen:

```
PVector topLeft = new PVector(255,0,0); // red
PVector topRight = new PVector(0,0,255); // blue
PVector bottomLeft = new PVector(255,255,0); // yellow
PVector bottomRight = new PVector(0,255,0); // green
```

You should then interpolate linearly between these colors to get a color for each pixel of the screen. Some Processing system variables that you might find useful for this are `height` and `width`, which store the values of the height and width of the display window respectively. The overall interpolation can be done by first interpolating the values of the very left, and right side of the screen:

```
Interpolate( topLeft, bottomLeft, height, leftSide );
Interpolate( topRight, bottomRight, height, rightSide );
```

Then for each row of the screen you can interpolate between the values of the left and right side of this row. This can be done in the for-loop in the Draw-function. Try to get the result shown in Figure 1 (left). After succeeding in this you can experiment with different colors of the corners to get some more intuition about what happens when colors are mixed/interpolated.

For this task, you might also like to investigate the use of Java HashMaps (https://www.w3schools.com/java/java_hashmap.asp).

Task 3: Starfield

In the previous part of the lab you have practiced interpolation and working with colors. In this part we will move ahead and start looking at geometry and in specific projections. We will also look at how we can update parameters within the framework to create an animated image. In specific we will create an implementation of the classic star field effect which have been used for things such as screen-blankers and famously for showing the Millennium Falcon's traveling through space. The effect is created by modeling a set of 3D points that moves through space (in time) towards the camera. If the camera is pointing along the z-axis then this will be the update equations for a point $p_i = (x_i, y_i, z_i)$:

$$\begin{aligned} x_t^i &= x_{t-1}^i \\ y_t^i &= y_{t-1}^i \\ z_t^i &= z_{t-1}^i - v \cdot dt \end{aligned} \tag{1}$$

where *v* is the velocity of the points and *dt* is the elapsed time between the two frames.

<u>Task 3.1:</u>

Initialise the starfield. To be able to use the update equations we need to initialize the points and determine the locations at time zero. Create a new global variable to store the locations of all stars:

```
ArrayList<PVector> stars = new ArrayList<PVector>( 1000 );
```

In this case we have created one thousand stars, represented as 3D points. Next you should initialize the location of every star at the first frame. If we assume that stars are randomly located in space, we can use the Java Random class to initialize the locations randomly. Here is some example code to generate a random number:

```
import java.util.Random;

Random rnd = new Random();

float x = rand.nextFloat();
```

This will produce a random number between zero and one. Create a for-loop in the beginning of the main function that loops through all stars and sets random positions for them as follows:

$$\begin{aligned} -1 \leq x \leq 1 \\ -1 \leq y \leq 1 \\ 0 < z \leq 1 \end{aligned} \tag{2}$$

<u>Task 3.2:</u>

Projection - Pinhole Camera field of view. Now when we have a created the set of 3D points we need to implement the rendering of the points on the screen. In order to do this we need to implement a simple pinhole camera that will provide the perspective effect when the points are moving towards the camera. To make things simple we assume that the camera is positioned

at the origin and looking in the direction of the z-axis. We assume that the x-axis and y-axis of the 3D coordinate system points in the same directions as those for the 2D screen coordinate system. However, we assume that the center of the 3D coordinate system is at the center of the screen having width W and height H. The projection from a 3D point (x_i, y_i, z_i) to a 2D screen coordinate (u_i, v_i) can then be written:

$$\begin{aligned} u^i &= f \cdot \frac{x^i}{z^i} + \frac{W}{2} \\ v^i &= f \cdot \frac{y^i}{z^i} + \frac{H}{2}. \end{aligned} \tag{3}$$

where f is the focal length of the camera. A good value for it is **f=H/2**. Then the **vertical field of view (VFOV) for the camera will be 90 degrees**. What is the resulting horizontal field of view? Try to calculate this.

Hint: You will use *arctan* in addition to the *aspect ratio* to find the horizontal field of view. Work it out on paper first. When you program it, be careful about converting between degrees and radians – check what types your math functions accept as parameters and return!

Useful Java functions: `Math.toDegrees(), Math.atan2().`

Task 3.3:

Implement the projection and drawing of all points in the Draw function. Before drawing the projected position/pixel of the stars as white you should make the background black.

Your implementation should look something like this:

```
ArrayList<PVector> stars = new ArrayList<PVector>(1000);


void draw()
{
        background(0);   //set background to black
        // calculate focal length f
        // calculate center point of camera projection
        PVector center = new PVector(width/2, height/2);
        for (int i = 0; i < stars.size(); i++) {
                // calculate u for this star from f
                // calculate v for this star from f
                //set the star position
        }
}
```

You should see a black background with white stars on the screen.

Task 3.4:

Add motion. It is now time to make things a bit more interesting by adding motion. We recommend that you create a new function:

```
void update();
```

to handle this. This function can be called each frame when starting to draw, i.e. the main loop of the program should look like:

```
void draw(){
	update();
	…
}
```

To measure the time between two frames you can use the function millis(). This function returns the number of milliseconds since starting the *Processing* sketch.

First create a global variable and initialize t to hold the current time:

```
int t = millis();
```

Then in the update() function you can compute the time since the last update by writing:

```
void update() {
	int t2 = millis();
	float dt = t2-t;
	t = t2;
}
```

The local variable dt will then hold the number of milliseconds that has passed since update() was last called. Use this and write code that updates the position of all stars according to *equation 1*. Choose a reasonable value for the velocity v. If the stars are outside of the volume specified in *equation 2* after the update they should be wrapped around to the other side, to make the illusion that the stars never end no matter far we travel:

```
for (int s = 0; s < stars.size(); s++) {
	 PVector star = stars.get(s);
	// Add code for update of stars
	…
	if (star.z <= 0) z +=1;
	if (star.z > 1) z -=1;
	…
}
```

If you have implemented the code above correctly, including moving the stars according to their velocity, you should see the stars moving towards you. You might notice a popping effect when the stars wrap around from the nearest to the farthest part of the volume. To make the transition smoother you can fade the brightness of the stars, i.e. let it depend on the distance to the camera. A physically motivated way to do this is to let the brightness be inversely proportional to the squared distance:

```
float c = 0.2*255/pow(stars.get(i).z,2);
```

This should be all the necessary steps to create the starfield effect. In order to pass this part of the lab you should show an implementation with a set of points moving towards the screen. Further, you should be able to explain and derive all the equations above. If you think this lab was too easy (there's always one!) you can also try to add motion blur to the starfield. That will look even cooler. But you'd be better off starting lab 2 instead, hotshot.